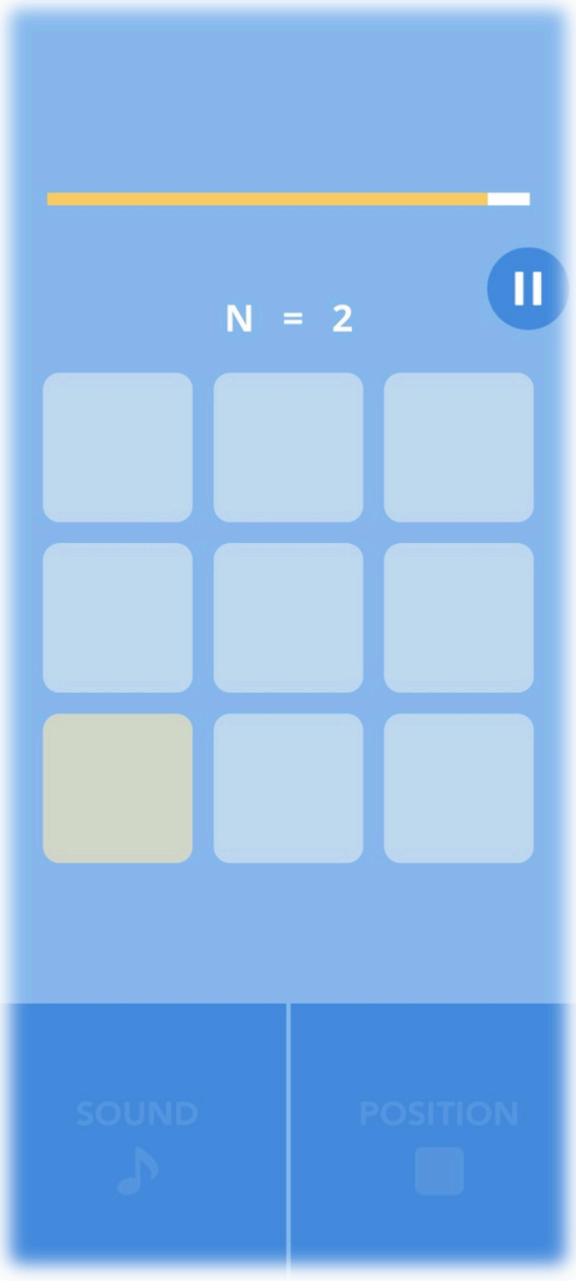
Lab B:1

Building a Native N-Back Mobile App

HI1033 / CM2023 · 25 okt. 2026 · @Jonas Willén

Aim

In this lab you will design and build a working N-back game for a mobile platform of your choice. You will practice graphics, touch events, animations, game-state logic, persistent storage, and audio and you will use AI tools (LLMs) as part of your development workflow.



The N-back task is a standard cognitive test used to measure working memory. A sequence of stimuli is presented one at a time; the player must press a button whenever the current stimulus matches the one from *n* steps earlier in the sequence.

N-back Background

The n-back task is a continuous performance task widely used in cognitive neuroscience to measure working memory capacity. It is also used as a training tool.

A sequence of stimuli is shown one at a time for example, a highlighted cell in a grid (visual) or a spoken letter (auditory). The player must press a button when the current stimulus matches the one from exactly *n* steps earlier. A dual n-back presents two independent sequences simultaneously (one visual, one auditory), and the player tracks both.

You can try an existing implementation at brainscale.net/dual-n-back/training.

General

This assignment is solved **individually**. You must pass this lab before you may continue to Lab B:2 and B:3.

Submit at least one day before your scheduled presentation. Your submission must contain four parts:

1. **Code:** The full project, structured in appropriate classes following MVVM (MVC is acceptable; a single god-class is not). Class, method, and field names must describe their purpose. Code must be documented.
2. **LLM process document:** A short document (free format, ~1 page) describing: which tools you used (Claude, Mistral, etc.); the context files you gave the LLM (attach `SPEC.md`, `STYLE.md`, or equivalent); two or three representative prompts that shaped the code and what they produced; and what the LLM got wrong that you had to correct.
3. **Test report:** A short document listing which parts of the logic you tested, which test cases you wrote, and the results. Attach the test files.
4. **Reflection** (~half a page): One thing that surprised you about working with AI, and one thing you would do differently next time.

At the oral examination you must be able to explain any line of code, regardless of whether you or an AI wrote it.

- The solution must be structured in appropriate classes following **MVVM** architecture (MVC is acceptable; a single god-class is not).
- Class, method, and field names must describe their purpose.
- Code must be documented so that another developer/you can understand it.
- You are encouraged to use LLMs (Claude, Mistral, ChatGPT, etc.) as coding tools. All AI assistance must be noted in your submission. At the oral examination you must be able to explain any line of your code.

Choose Your Platform

You implement the app on **one** platform of your choice. Pick it before you start and stay with it throughout the lab.

Platform	Language	UI framework	Notes
iOS	Swift	SwiftUI	Xcode
Android	Kotlin	Jetpack Compose	Android Studio
Cross-platform	TypeScript	React Native (Expo)	Works on both iOS and Android from one codebase

All three platforms must fulfill the same functional requirements. Platform-specific hints are listed at the end of this document.

Basic Requirements (1 point)

Create a single n-back application with at least **two screens**:

1. **Home screen** — shows the current high score and lets the player choose between visual stimuli, auditory stimuli, or both. Displays the current settings (value of n, event duration, number of events per round).
2. **Game screen** — shows the 3×3 grid for visual stimuli and/or plays spoken letters for auditory stimuli. Displays live game state (current event number, correct responses so far). Includes a button to register an n-back match and gives non-offensive animated feedback on an incorrect press.

- **The sequence logic** (match rate ~20–40 %, guarantee that the first *n* events can never be matches) must be implemented by you not delegated entirely to a library.

- **Unit tests** for at least three of the following: sequence generation, match-rate enforcement, first-*n* guarantee, double-press prevention, scoring formula. Tests must be runnable and pass on the submitted code.

Higher Level Requirements (2 points)

For full marks, implement **dual n-back** (visual and auditory simultaneously) and at least **three** of the following:

- **Settings screen** (persistent): number of events per round, event duration, value of *n*, grid size (e.g. 5×5), number of distinct spoken letters.
- **Score history**: players save a score under a name; a separate screen lists all scores sortable by name, score, and date. Backed by a local database.
- **Responsive layout**: the app works correctly in portrait and landscape, and on both phone and tablet screen sizes.
- **Accessibility**: minimum touch-target size met on the match button; TTS fallback message when no voice is available.

Detailed Requirements

All of the following are mandatory to pass.

- Visual stimuli displayed in a 3×3 grid; one cell highlighted per event.
- Auditory stimuli as spoken letters (a fixed, phonetically distinct set).
- Controls to start a new round and to register an n-back match.
- Home screen shows current settings: stimulus type, *n*, event duration, events per round.
- Game screen shows live state: current event number and number of correct responses.
- Non-offensive animated feedback on an incorrect match press (e.g. color or size animation on the button).
- A match press is ignored before the round starts, after it ends, and during the first *n* events.
- Multiple match presses on the same event are ignored after the first.
- The round timer is cancelled cleanly when the round ends or the player leaves the screen mid-round.

AI & LLM Usage

You are encouraged to use LLMs such as Claude, Mistral, ChatGPT, or similar as coding assistants throughout this lab. This reflects real-world professional practice.

What you must do:

- Include a short section in your submission stating which parts were AI-assisted and which prompts or tools you used.
- Be able to explain any line of code at the oral examination, regardless of whether you or an AI wrote it.
- Verify and test all AI-generated code yourself before submitting.

Recommended workflow: Create a `SPEC.md` describing the lab requirements and your platform choice, a `STYLE.md` with your architecture decisions, and a `CONTEXT.md` with course-specific constraints (single-file preferred, no external libraries beyond the platform SDK). Feed these to the LLM at the start of each session to get consistent, on-target output.

What is not allowed: Submitting code you cannot explain, or copying another student's solution (AI-generated or not).

Hints

iOS (Swift / SwiftUI)

- Use `AVSpeechSynthesizer` for text-to-speech; initialise the engine once and speak per event.
- Store scores in **Core Data**; use `NSSortDescriptor` for sorted queries.
- Store settings in **UserDefaults**.
- Test on both iPhone and iPad simulators to validate layout.

Android (Kotlin / Jetpack Compose)

- Use `TextToSpeech` for audio; initialise in the ViewModel and release in `onCleared()`.
- Store scores in a **Room** database with a `@Query` that returns results sorted.
- Store settings with **DataStore<Preferences>**.
- Test on both phone and tablet AVDs.

Cross-platform (React Native / Expo)

- Use `expo-speech` for text-to-speech.
- Store scores in **SQLite** via `expo-sqlite`; write a query that returns rows sorted.
- Store settings with `AsyncStorage` or `expo-secure-store`.
- Test on both iOS and Android simulators; check portrait and landscape with `useWindowDimensions`.

Oral Examination Questions

At your presentation you will be asked to open your code and answer questions from this list. You do not need to memorize answers — but you must be able to find the relevant code and explain how it works. The opening question is always **question 26**.

Stimuli & sequence logic

1. Walk through the function that generates the N-back sequence. Where is it, and what does it return? How to you test it?
2. How do you enforce a ~20–40 % match rate? Where in the code is that probability decided for each event? How to you test it?
3. How do you guarantee that the first *n* events can never be matches? How to you test it?
4. How does the generated sequence drive both the visual event (which grid cell lights up) and the audio event (which letter is spoken)? Trace one stimulus from generation to output.

Game lifecycle

5. Describe the game state machine (idle / running / finished) and every transition between states.
6. Trace one full event tick end-to-end: advance sequence → show stimulus → speak letter → reset match flag → update HUD.
7. How does the app detect that a round is over, and what cleanup runs at that point?
8. How does the app guard against a match press before the round starts, after it ends, and during the first *n* events? How to you test it?
9. How does the app prevent a second match press on the same event? How to you test it?

Scoring & feedback

10. Where are hits, misses, and false alarms counted both on a press and on a silent tick?
11. What is the score formula? Defend the weights and explain how a negative score is prevented.
12. Where is the round timer cancelled, and what happens if the player leaves the screen mid-round?
13. How is wrong-guess feedback triggered, and why is it non-offensive?
14. Where is the HUD (current event / correct so far) updated?

Architecture

15. Show the observable-state wiring (StateFlow / @Published / useState): where is state declared and where is it subscribed?
16. How does navigation between screens work, and why does the ViewModel not navigate itself?
17. Where in the code is unit-testable logic separated from the UI layer? What made that separation easy or hard to test?

Audio

18. Where is the TTS engine initialized, and where does each letter get spoken? Why does speech only happen after a user gesture?
19. Where is the letter set defined, and why those specific letters (phonetically distinct)?
20. What happens when no voice is available on the device?

Persistence & platform

21. Walk through one settings write and one read (where in the code, what API).
22. Walk through the score entity/table and one sorted query. Why a database here rather than key-value storage?
23. What runs off the main thread and how (dispatcher / background context / async)?

UI & platform behavior

24. How does the grid stay square across phones, tablets, portrait, and landscape?
25. How is the minimum touch-target size met for the match button, and how are safe areas handled?

AI & process

26. Which parts did you write yourself and which were AI-generated? Be ready to explain any line you are asked about.

Testing

27. Which three (or more) components did you write unit tests for? Walk me through why you chose those over others.
28. Your app must prevent a double-press (registering a second tap before the next stimulus arrives). Show the code that enforces this and the test that would catch a violation if the guard were removed.
29. Run one of your unit tests live right now. Explain what the assertion checks and describe a change to the production code that would make it fail.
30. Did any test yours or LLM-generated catch a real bug? Describe what failed and how you fixed it.